

qtree_NLJoin_优化

qtree 现有NLJoin

- qtree 的 NLJoin 创建过程中，没有 join_cond. 将所有的 filter 都放到了 other_filter 中。
- 当前 qtree 中 NLJoin 的实现过程概述：
 1. 初始化阶段判断如果没有 join_cond，将 kind 置为 cross。即使之前是 right、left
 2. joinBlockImplCross(): 多层循环，全量，所有列生成笛卡尔积
 3. executeOtherCondition(pro_thd->m_result_block)，生成 filter
 4. remove_duplicate_index(): 利用 filter 进行过滤，并未考虑 outter join 的后续情况
 5. 结果 push 到 m_output_stream 中

- qtree 中的 NLJoin 并没有相应的 outter Join 逻辑，所以可以参考下面整理的 sr 中的实现逻辑。
- 单纯的 InnerJoin，也有可以借鉴的地方（不使用循环，而是针对左右表的大小选择不同批量插入方式）

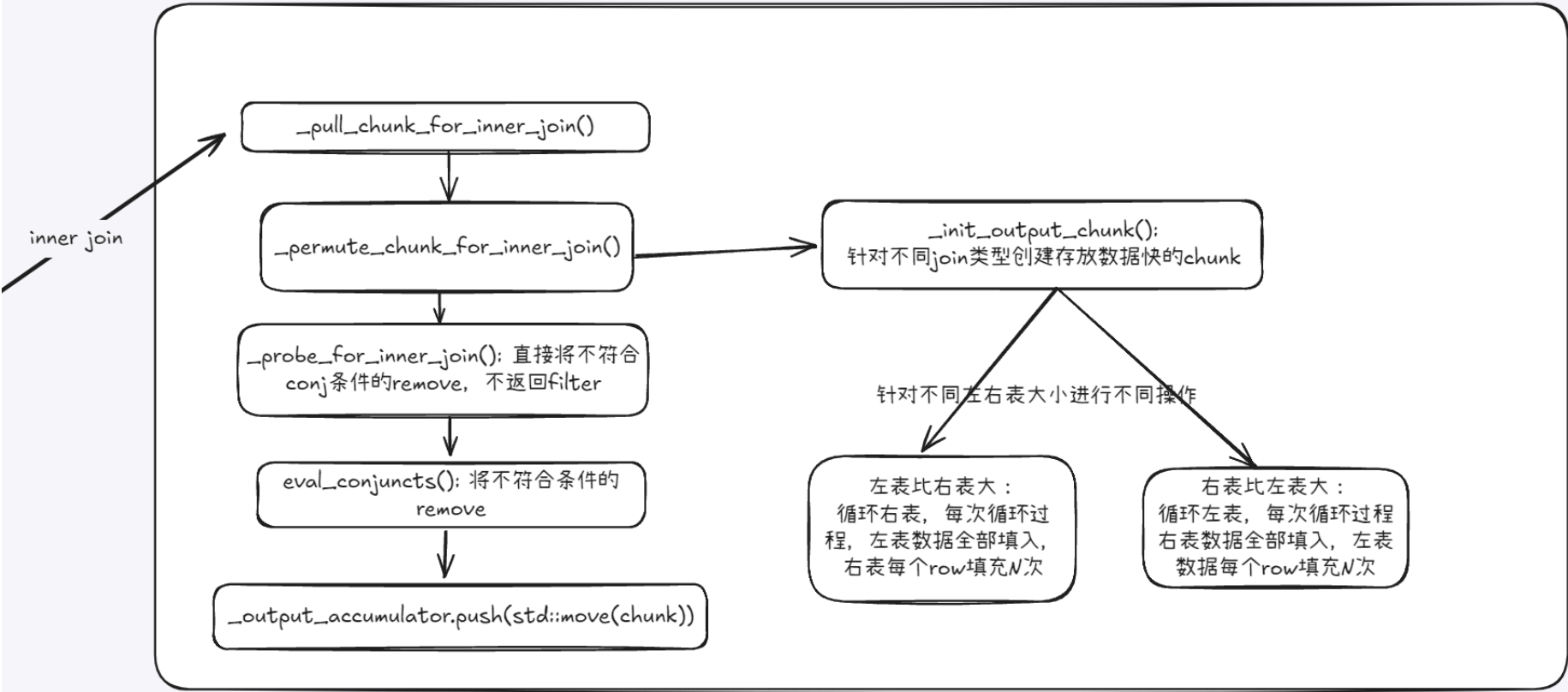
StarRocks 中 NLJoin 实现逻辑

build

- build 过程并没有需要借鉴，sr 中 build 和 probe 分成两个算子，通过共享 context 来实现右表数据的传输。qtree 中不需要这种操作。
- 当前算子行数是 4096
- build 的结果是：n 个 complete_chunk + m 个 incomplete_chunk
 - incomplete_chunk() 就是 chunk.rows < chunk.size(): 4096
 - n(0,)
 - m(0, 1)，n 为0时 m 必为1

probe

- InnerJoin：InnerJoin 与 qtree 实现逻辑一致，没有什么区别，生成笛卡尔积，进行条件过滤。
 - 其中可以考虑借鉴的：不使用双层循环，批量插入行



- leftJoin：qtree 中的 NLJoin 并无 leftJoin 实现
 - 生成笛卡尔积，根据不同 build_chunk 个数，笛卡尔积的 chunk (也是 4096 行)有两个情况：
 - 一个左 chunk 中的行 *4096 与右边一个 build_chunk 组成笛卡尔积 chunk
 - 右 build_chunk (n 为0 m 为1情况)所有行重复多次，左 Probe_chunk 多行重复

- probe 操作：
 - 先执行 join 条件**过滤**，返回 filter。
 - 如果右表为空，res_chunk 中右表的列填充 null 值返回
 - 右表不为空，根据上述笛卡尔积两种情况进行判断是否有未匹配上的行，填充 null 值：
 - 第一种情况，当前 probe_row 的笛卡尔积可能存在多个 chunk 需要等所有 chunk 都判断是否匹配上。
 - 第二种情况，需要针对当前 chunk 中不同 probe_row 进行判断（第二种情况当前 chunk 中会有多行 probe_row 的笛卡尔积）
 - 执行 eval_conjunction 过滤，这里过滤初步推测是where条件过滤
 - 放入 accumulator 中，acc 达到一定行数之后返回结果
-

- left anti Join
 - 生成笛卡尔积，与上述情况一致
 - probe 操作：
 - 如果右表是空，就直接 return probe_chunk 的全部行
 - 先执行 join 条件**但不过滤**，返回 filter
 - 之后针对当前 probe_row，判断是不是没有匹配上，**如果没有匹配上置 filter 当前 probe_row 的一行为1**。同样是两种情况
 - 第一种情况，当前 probe_row 的笛卡尔积可能存在多个 chunk 需要等所有 chunk 都判断是否匹配上。
 - 第二种情况，需要针对当前 chunk 中不同 probe_row 的 filter 进行判断
 - 中途可能会遇到当前 probe_row 匹配上的时候，如果匹配上了，需要让当前 probe_row 未完成的笛卡尔积停止。不再进行当前 row 的 probe 操作
 - 执行 filter 过滤
 - 执行 eval_conjunction
 - 放入 acc 中，acc 达到一定行数之后返回结果
-

- left semi Join：
 - 生成笛卡尔积，与上述情况一致
 - probe 操作：
 - 先执行 join 条件**但不过滤**，返回 filter
 - 针对当前 probe_row，判断是不是没有匹配上，**如果匹配上置 filter 当前 probe_row 的一行为1**。同样是两种情况
 - 当前行匹配上，剩余未进行的笛卡尔积操作不用进行了
 - 执行 filter 过滤
 - 执行 eval_conjunction
 - 放入 acc 中，acc 达到一定行数之后返回结果
-

- rightJoin：
 - 生成笛卡尔积，与上述情况一致
 - probe 操作：
 - 执行 join 条件并**过滤**，返回 filter
 - rightJoin 需要保留一个当前线程的 filter_flag，与右表大小一致，来记录当前线程中右表的匹配情况
 - 针对当前 probe_row，匹配情况记录到 filter_flag 中。
 - 执行 eval_conjunction
 - 放入 acc 中，acc 达到一定行数后返回结果（这里是匹配上的数据）
 - 当前线程执行结束时会将当前线程的 filter_flag 与全局的 match_flag 左表或操作。所有线程做完这个操作之后，右表匹配情况就到了 match_flag 中
 - 进行未匹配的填充 null 值操作。将未匹配的填充 null 之后返回结果

StarRocks 中 NLJoin 代码细节

probe

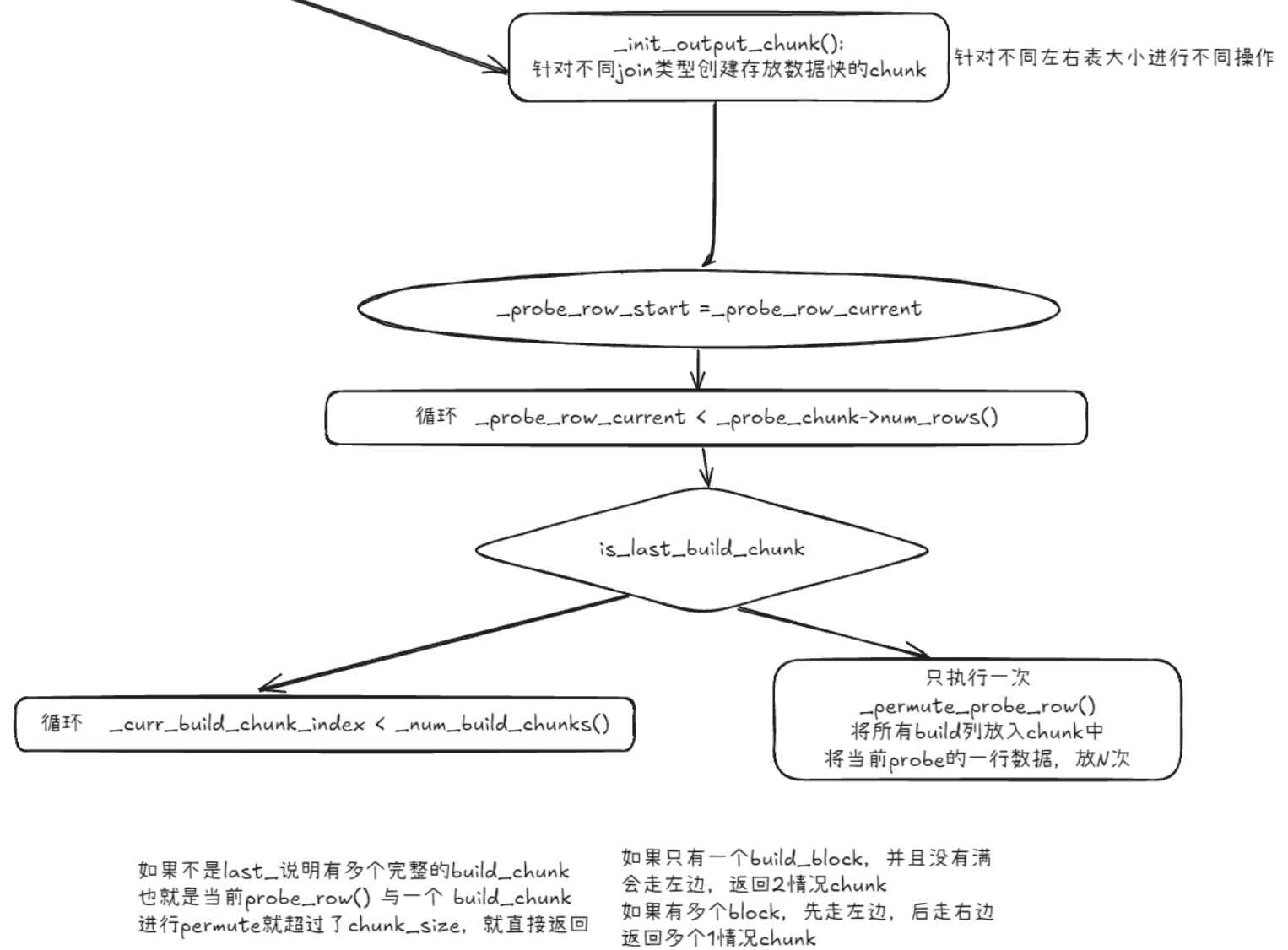
- 实现过程：
 - 左右表首先计算笛卡尔积（全量，也未进行非 join 列的去除），直到 4096 rows。
 - Apply join_cond filter、other filter，添加到 output buffer 中。
 - 针对 join_cond filter 的执行结果，要进行保存，针对不同类型的 join 做不通过处理

- StarRocks 中实现的 join 类型不包括：right semi、right anti
- _init_output_chunk 的过程 innerJoin 和 otherJoin 使用的函数没有区别
 - 针对 inner Join 不做特殊处理，outter join 的外表列在 output_chunk 中需要置为 nullable()
- InnerJoin：
 - _permute_chunk_for_inner_join()
 - _init_output_chunk
 - 针对左右 trunk 行数不同，选择不同的笛卡尔积方式
 - 以左块行数 < 右块行数为例，会针对左块进行循环，每次取一行，右块数据直接拷贝列到 output_chunk 中，左块当前行放入右块行数次。（可以添加到当前 qtree 中，qtree 目前为两层循环，一个值一个值的插入）
 - _probe_for_inner_join()
 - 将不符合 join_cond 的列移除
 - eval_conjunction()
 - 将不符合 join_filter 的列移除，这里的条件根据代码来看是非连接条件的过滤条件，类似，非等值 join 中的条件

-
- other join 有一些与 InnerJoin 不同的函数，这里先介绍对应函数，再介绍不同 join 类型的执行流程

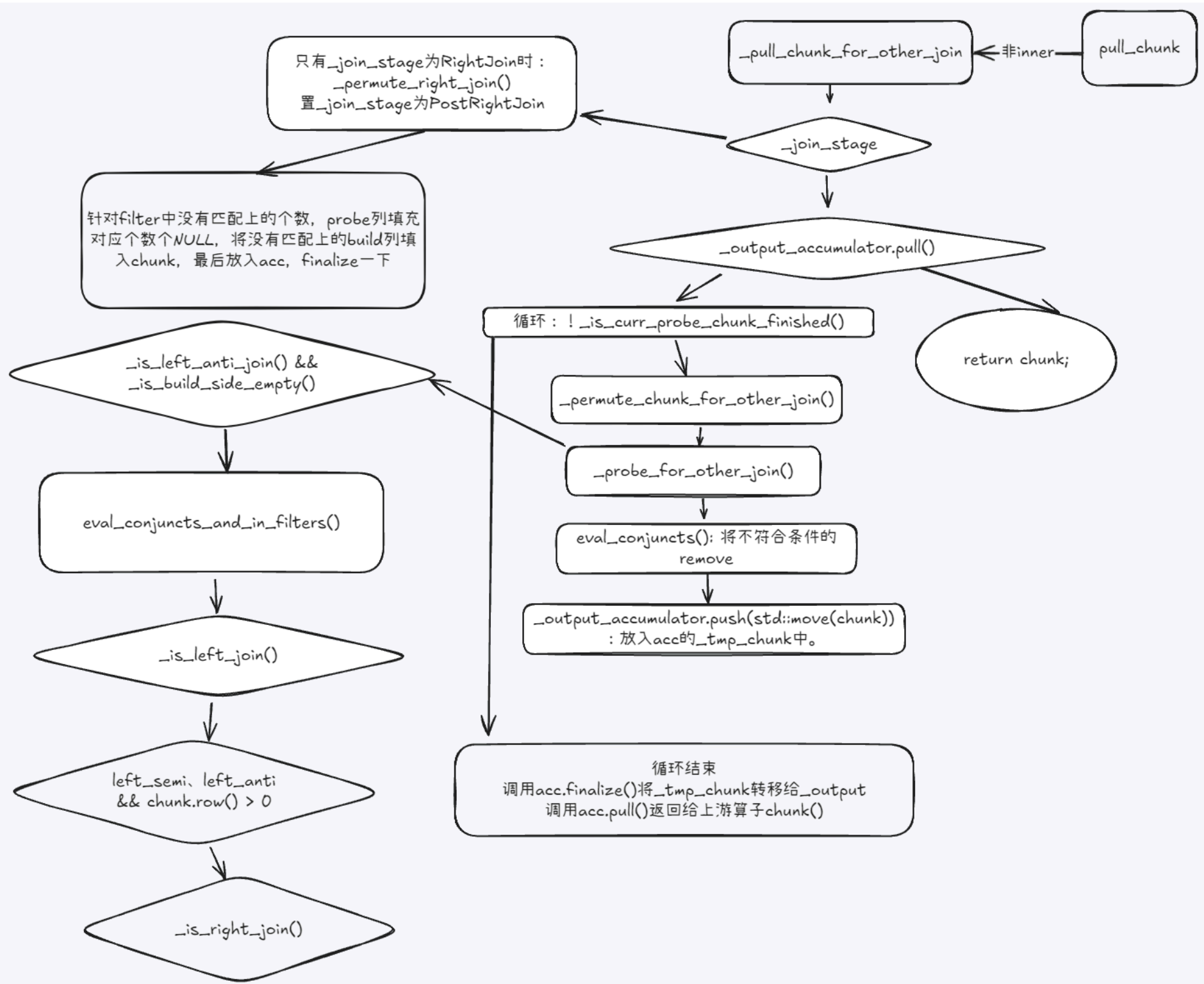
1. _permute_chunk_for_other_join()：也是进行笛卡尔积，与 innerjoin 不同，other join 需要考虑之后 probe 阶段的操作
 - _init_output_chunk
 - 由于 build 结果会有两情况，只有一个 build_chunk 时行数 <=4096 行，或者多个 build_chunk 时最后一个 chunk 行数 <=4096 行
 - 只有一个 build_chunk 时：
 - 会将当前 probe_row 重复 build_chunk 行数次放入 output_chunk 中，build_chunk 直接填入 output_chunk
 - 循环，直到当前 output_chunk 行数 > 4096
 - 多个 chunk 时：
 - 会将当前 probe_row 重复 4096 行数次放入 output_chunk 中，build_chunk 直接填入 output_chunk，直接返回
 - 输出时候会将当前 probe_row 行数记到 _probe_row_start 中
 - 如果是最后一个 chunk，可能不够 4096 行，直接返回。
 - 也就是当前函数的返回只有两个结果：
 - One probe row with one build_chunk
 - multi probe rows with one single build_chunk
 - 针对 probe 阶段提供了两个变量
 - 输出时候会将当前 probe_row 行数记到 _probe_row_start 中
 - _probe_row_finished 是判断当前行有没有 probe 结束
 - left join 需要等当前行结束之后在判断是否匹配上
 - left anti 如果提前遇到了匹配，当前行也就 probe 结束了，笛卡尔积阶段就提前结束。

- 结果只有：
1. One probe row with one build_chunk
 2. multi probe rows with one single build_chunk



2. `_permute_left_join(chunk, probe_row_index, probe_rows)`: 从 index 开始, 将 `_probe_chunk` 的数据填入 chunk , `_build_chunk` 对应位置填充 `probe_rows` 个 null 值

- `otherJoin` 执行流程图 (图中 `_probe_for_other_join()` 的判断逻辑为真的执行流程未标出) :



• left join :

1. `_permute_chunk_for_other_join()` 见上面

2. `_probe_for_other_join()` : 进行 join 条件的匹配针对不同 join 类型选择不同的策略 (是否在匹配时过滤: left anti、left semi 不进行过滤)

- 如果有 `join_conjunction_cond` 并且当前 chunk 不为空, 执行 `eval_conjuncts_and_in_filters()`, 会返回 filter: left join 执行过滤, 将不符合的数据剔除,
- 如果右表为空执行: `_permute_left_join(chunk, 0, _probe_chunk->num_rows());`, 从 0 开始填入 probe_chunk 函数个 null, 因为右表为空的话, 左表的同 chunk 不管多少个值, 经过笛卡尔积之后都会在同一 chunk 中, 不会出现重复或者丢失的值
- 如果有 filter, 说明右表不为空, 同样两种处理逻辑:
 - 只有一个 build_chunk 时: 当前 chunk 中的情况是 multi probe rows with one single build_chunk, 需要利用 `_probe_row_start + i / num_build_rows` 来计算出当前 probe_row 所处位置, `size_t i = 0; i < filter->size(); i += num_build_rows`
 - ex: build_chunk 为 1024 行, 当前 chunk 中有 4 行 probe_chunk 的值, 4 行分别重复 1024 次, 最后为 4096 次, `probe_row_start = 0`, 下一行的开始位置就是 `0 + num_build_rows = 1024`.
 - 如果 filter 从 0 - 1023 中没有不为 0 的, 说明第 0 行没有匹配上, 执行 `_permute_left_join(chunk, probe_row_index, 1)`
 - 如果多个 build_chunk 时: 当前 chunk 中的情况是 One probe row with one build_chunk, 也就是当前 probe_row 有没有匹配到只需要看当前 4096 行的 filter 有没有 0, 记录到 `_probe_row_matched` 中, 利用 `_probe_row_finished` 判断当前行是否与所有 build_chunk 进行了笛卡尔积并进行到了当前环节。如果 `!_probe_row_matched && _probe_row_finished` 说明没有匹配上, 执行 `_permute_left_join(chunk, _probe_row_current, 1)`

3. `eval_conjuncts()` 同上

• left anti join: anti 需要返回所有不匹配的左表, 不返回右表

1. `_permute_chunk_for_other_join()` : 同上

2. `probe_for_other_join()` :

- 如果右表是空，执行 `_permute_left_join(chunk, 0, probe_chunk.rows())` 将结果集右边填充null（感觉不是很需要，`left anti` 的结果列在投影列中的输出结果与右边填充null的结果一样）
 - 如果有 `join_conjunction_cond` 并且当前 `chunk` 不为空，执行 `eval_conjuncts_and_in_filters()`，会返回 `filter: left anti join` 不执行过滤
 - 如果当前 `filter` 为空，需要创建 `filter`
 - 遍历当前 `chunk`，针对当前 `probe_row` 利用同样的方式针对不同 `build_chunk` 个数判断出当前 `probe_row` 的 `row_start` 和 `row_end`，利用 `SMID` 获取 `filter` 在 `(row_start, row_end)` 中第一个非0位置 `first_matched`
 - 将 `filter` 的 `(row_start, row_end)` 全部填为0
 - 如果 `first_matched == end`，当前行没有匹配上
 - 如果只有一个 `build_chunk`：当前 `probe_row` 全部笛卡尔积都在这个 `chunk` 中；直接置 `filter[start] = 1` 保留当前保留 `probe_row`
 - 如果多个 `build_chunk` 但是 `_probe_row_finished = true` 就置 `filter[start] = 1`，保留当前 `probe_row`
 - 如果 `first_matched != end`，当前保留 `probe_row` 匹配上，置 `_probe_row_finished = true`，不用针对该行进行笛卡尔积操作了
 - 执行 `filter` 过滤
3. `eval_conjuncts()` 同上
- `left semi join: left_semi` 需要返回所有匹配的左表，但是不返回右表
 1. `_permute_chunk_for_other_join()`：同上
 2. `probe_for_other_join()`
 - 如果有 `join_conjunction_cond` 并且当前 `chunk` 不为空，执行 `eval_conjuncts_and_in_filters()`，会返回 `filter: left semi join` 不执行过滤
 - 如果当前 `filter` 为空，需要创建 `filter`
 - 遍历当前 `chunk`，针对当前 `probe_row` 利用同样的方式针对不同 `build_chunk` 个数判断出当前 `probe_row` 的 `row_start` 和 `row_end`，利用 `SMID` 获取 `filter` 在 `(row_start, row_end)` 中第一个非0位置-
 - 将 `filter` 的 `(row_start, row_end)` 全部填为0
 - 如果 `first_matched <= end`，当前行匹配上，置 `filter[first_matched] = 1`，保留该行。置 `_probe_row_finished`，表示当前行匹配上，剩余未进行的笛卡尔积操作不用进行了
 - 执行 `filter` 过滤
 3. `eval_conjuncts()` 同上
 - `right Join`
 - 如果是 `right Join` 的话，会创建一个与 `build_chunk` 的所有行数一样大的 `_self_build_match_flag` 来存储当前 `prober` 线程中右表匹配情况。
 - 1. `_permute_chunk_for_other_join()`：同上
 - 2. `probe_for_other_join()`
 - `eval_conjuncts_and_in_filters()`：`right Join` 执行过滤
 - 两种 `build_chunk` 个数情况下判断当前 `probe_row` 是否有匹配上，与 `left Join` 中执行逻辑类型，但是是将匹配结果放入 `_self_build_match_flag` 中
 - 3. `eval_conjuncts()` 同上
 - 4. 将结果放入 `acc` 中。
 - 5. 当前线程执行结束，会调用相应接口，接口中：
 1. 会将 `_self_build_match_flag` 与 `_shared_build_match_flag` 做或操作将当前线程的匹配情况填充到对应位置。
 2. `_num_post_probers++`，表示已经完成的个数。如果已经完成的 `prober` 个数与创建的个数一样（其他类型 `join` 也有这个变量，但是由于没有实际作用，就没有阐述）。表示所有 `prober` 结束，执行下一步操作
 - 6. 所有 `prober` 结束，之后，会利用 `_shared_build_match_flag` 判断所有未匹配到的行，将对应行填充到 `res_chunk` 中，`res_chunk` 执行 `eval_conjuncts()`。
 - 7. 将结果放入 `acc` 中。